

Sistemas Operativos: Práctica 3

Shaofan Xu

Javier Maria Santa

8 de mayo de 2026

Índice

1. Introducción	3
2. Requisitos cumplidos y no cumplidos	3
2.1. Requisitos cumplidos	3
2.2. Requisitos no cumplidos	4
3. Pruebas de Funcionamiento	4
3.1. Entorno de ejecución	4
3.2. Pruebas de Funcionamiento	4
3.2.1. Casos normales	4
3.2.2. Pruebas con valgrind	5
3.2.3. Casos de errores	7
4. Conclusión y Respuesta a las cuestiones del enunciado	7
5. Otros aspectos	8

1. Introducción

En esta práctica el objetivo es ampliar la simulación de minería de blockchain introduciendo mecanismos avanzados de comunicación entre procesos (IPC) y memoria compartida. En concreto, se centraliza la salida de los mineros a través de un sistema de monitorización basado en colas de mensajes y un esquema productor-consumidor.

2. Requisitos cumplidos y no cumplidos

2.1. Requisitos cumplidos

Se han implementado correctamente los siguientes objetivos principales de la práctica, correspondientes a los apartados a, b y c:

- **Sistema de Monitorización (Apartado a):** Se ha desarrollado el ejecutable `monitor`, el cual exige estar arrancado antes que cualquier minero. Este consta de dos procesos que gestionan la salida unificada:
 - **Comprobador (Padre):** Recibe los bloques ganadores a través de una cola de mensajes POSIX (`/mq_monitor`). Antes de procesarlos, verifica su validez comprobando que `pow_hash(solucion) == target`. Si es válido, actúa como *Productor* e introduce el bloque en memoria compartida.
 - **Monitor (Hijo):** Actúa como *Consumidor*, leyendo los bloques de la memoria compartida e imprimiéndolos por pantalla respetando los *lags* de tiempo exigidos en la invocación.
- **Esquema Productor-Consumidor IPC:** La comunicación interna del Monitor se ha sincronizado empleando un segmento de memoria compartida (`shm_open` y `mmap`) que alberga un búfer circular. El acceso concurrente a dicho búfer está protegido mediante tres semáforos POSIX sin nombre (`mutex`, `empty` y `full`), sustituyendo así a los semáforos con nombre de prácticas anteriores.
- **Adaptación del Minero (Apartado b):** El proceso Minero envía un mensaje con su `pid`, ronda, `target` y solución por la cola al proclamarse ganador de una votación. Asimismo, cuando el tiempo del minero expira y detecta que es el último en salir, envía un bloque especial de terminación (`{-1, -1}`) por la cola.
- **Adaptación del Minero (Apartado c):** Se ha sustituido el uso de los ficheros por la memoria compartida, con el que hemos añadido campos necesarios para los mineros como el recuento de los votos y etcétera. Manteniendo el uso de fichero en el proceso Registrador.
- **Limpieza controlada de recursos:** Al recibir el bloque de finalización del último minero, el Comprobador intercepta el mensaje y se encarga de realizar las llamadas `mq_unlink`, `shm_unlink`, `sem_destroy` y `sem_unlink`, cerrando a su vez a su proceso hijo. Esto garantiza que no existan fugas de memoria en las estructuras del sistema operativo, como se ha comprobado en los test con Valgrind.

2.2. Requisitos no cumplidos

Todos los requisitos obligatorios especificados en el enunciado han sido implementados y funcionan correctamente.

3. Pruebas de Funcionamiento

3.1. Entorno de ejecución

Las pruebas y mediciones se han llevado a cabo bajo las siguientes condiciones de hardware y software para garantizar la consistencia de los resultados:

- Sistema Operativo: WSL2 (Ubuntu) sobre Windows 11.
- Hardware: 16 GB de memoria RAM.
- Herramienta de medición: Comando time de la shell de Linux.

3.2. Pruebas de Funcionamiento

Se han ejecutado diversas pruebas para asegurar el correcto funcionamiento del programa.

3.2.1. Casos normales

Se ejecutó el script de pruebas proporcionado arrancando el monitor y múltiples mineros de forma concurrente (`./monitor 500 500` y varios `./miner 15 4`). En nuestro caso se puede probar con:

```
chmod +x test.sh
./test.sh
```

Salida obtenida (Traza representativa): Al finalizar, se observa la secuencia correcta de bloques validados, la votación y la salida escalonada de los mineros sin generar interbloques:

```
[Comprobador] Iniciado correctamente. Esperando bloques...
Miner 17532 added to system
Miner 17533 added to system
Miner 17534 added to system
Winner 17532 => [ Y Y ] => Accepted
Solution accepted: 00000000 --> 08916551
Winner 17533 => [ Y Y ] => Accepted
Solution accepted: 08916551 --> 05970796
Winner 17534 => [ Y Y ] => Accepted
Solution accepted: 05970796 --> 00937352
Winner 17533 => [ Y N ] => Rejected
Winner 17534 => [ Y Y ] => Accepted
Solution accepted: 00937352 --> 01835314
Winner 17533 => [ Y N ] => Rejected
Winner 17534 => [ Y Y ] => Accepted
Solution accepted: 01835314 --> 09331340
Winner 17532 => [ Y N ] => Rejected
Winner 17534 => [ Y Y ] => Accepted
Solution accepted: 09331340 --> 02986686
```

```

Winner 17532 => [ Y N ] => Rejected
Winner 17534 => [ Y Y ] => Accepted
Solution accepted: 02986686 --> 07299854
Winner 17532 => [ Y N ] => Rejected
Winner 17534 => [ Y Y ] => Accepted
Solution accepted: 07299854 --> 03981310
Miner 17532 exited system. Remaining miners: 2
Current miners in system:
  17533
  17534
Miner 17533 exited system. Remaining miners: 1
Current miners in system:
  17534
Miner 17534 was the last one. System cleaned.

```

3.2.2. Pruebas con valgrind

Para verificar el correcto funcionamiento de los códigos y que no haya fuga de memorias hemos utilizado el herramienta **valgrind** para ello (hemos proporcionado comandos en **makefile** para ejecutar **make valgrind_monitor** y **make valgrind_minero**).

Usando los siguientes comandos arrancando un programa de **monitor** y varios de **minero**:

Salida obtenida (Traza representativa) de Monitor:

```

valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes --trace-children=
yes ./monitor 500 500
==26836== Memcheck, a memory error detector
==26836== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==26836== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==26836== Command: ./monitor 500 500
==26836==
[Comprobador] Iniciado correctamente. Esperando bloques...
Solution accepted: 00000000 --> 08916551
Solution accepted: 08916551 --> 05970796
Solution accepted: 05970796 --> 00937352
Solution accepted: 00937352 --> 01835314
Solution accepted: 01835314 --> 09331340
Solution accepted: 09331340 --> 02986686
Solution accepted: 02986686 --> 07299854
Solution accepted: 07299854 --> 03981310
Solution accepted: 03981310 --> 08021305
Solution accepted: 08021305 --> 04673876
Solution accepted: 04673876 --> 02013491
Solution accepted: 02013491 --> 06435300
Solution accepted: 06435300 --> 03077079
Solution accepted: 03077079 --> 05218179
Solution accepted: 05218179 --> 00787429
Solution accepted: 00787429 --> 07952220
Solution accepted: 07952220 --> 02000011
Solution accepted: 02000011 --> 09327418
Solution accepted: 09327418 --> 08405727
Solution accepted: 08405727 --> 01457266
Solution accepted: 01457266 --> 01446753
Solution accepted: 01446753 --> 07216332
Solution accepted: 07216332 --> 08619049
Solution accepted: 08619049 --> 03117528

```

```

==26844==
==26844== HEAP SUMMARY:
==26844== in use at exit: 0 bytes in 0 blocks
==26844== total heap usage: 1 allocs, 1 frees, 1,024 bytes allocated
==26844==
==26844== All heap blocks were freed -- no leaks are possible
==26844==
==26844== For lists of detected and suppressed errors, rerun with: -s
==26844== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
==26836==
==26836== HEAP SUMMARY:
==26836== in use at exit: 0 bytes in 0 blocks
==26836== total heap usage: 1 allocs, 1 frees, 1,024 bytes allocated
==26836==
==26836== All heap blocks were freed -- no leaks are possible
==26836==
==26836== For lists of detected and suppressed errors, rerun with: -s
==26836== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

```

Salida obtenida (Traza representativa) de Minero:

```

valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes --trace-children=
yes ./miner 15 4
==804048== Memcheck, a memory error detector
==804048== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==804048== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==804048== Command: ./miner 15 4
==804048==
Miner 804048 added to system
Winner 804048 => [ Y Y Y ] => Accepted
Winner 804048 => [ Y Y Y ] => Accepted
Winner 804048 => [ Y Y N ] => Accepted
Winner 804048 => [ N N N ] => Rejected
Winner 804048 => [ Y N ] => Rejected
Winner 804048 => [ Y N ] => Rejected
Winner 804048 => [ Y Y ] => Accepted
Winner 804048 => [ Y ] => Accepted
Winner 804048 => [ Y ] => Accepted
Miner 804048 was the last one. System cleaned.
==804058==
==804058== HEAP SUMMARY:
==804058== in use at exit: 0 bytes in 0 blocks
==804058== total heap usage: 18 allocs, 18 frees, 41,112 bytes allocated
==804058==
==804058== All heap blocks were freed -- no leaks are possible
==804058==
==804058== For lists of detected and suppressed errors, rerun with: -s
==804058== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
==804048==
==804048== HEAP SUMMARY:
==804048== in use at exit: 0 bytes in 0 blocks
==804048== total heap usage: 41 allocs, 41 frees, 3,706 bytes allocated
==804048==
==804048== All heap blocks were freed -- no leaks are possible
==804048==
==804048== For lists of detected and suppressed errors, rerun with: -s
==804048== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

```

Nota sobre los votos negativos (N):

Durante la ejecución del programa, es posible observar que se registran votos negativos (N), incluso sin contar las rondas donde se ha forzado el envío de una solución errónea. Este comportamiento es totalmente normal, se debe a que, en el momento de la votación, algún minero puede estar validando la solución utilizando el `target` de una ronda anterior que aún no ha actualizado.

3.2.3. Casos de errores

Se probaron situaciones anómalas como:

- **Falta de argumentos:** Al ejecutar `./miner` o `./monitor` sin los parámetros obligatorios, el sistema atrapa el error inmediatamente, muestra la sintaxis correcta de uso (`Usage...`) y aborta la ejecución de forma segura.
- **Límite de mineros:** Se ha controlado la entrada al sistema para que no exceda el `MAX_MINERS` definido (en nuestro caso 100).

4. Conclusión y Respuesta a las cuestiones del enunciado

Pregunta d) En base al funcionamiento del sistema anterior:

- **¿Es necesario un sistema tipo productor-consumidor para garantizar que el sistema funciona correctamente si el retraso de Comprobador es mucho mayor que el de Minero? ¿Por qué?**

Sí, es necesario. Si el `Comprobador` tiene un retraso (lag) mucho mayor que el de los `Mineros`, entonces los mineros generarán ganadores más rápido de lo que el `Comprobador` puede verificarlos. En este caso, el modelo productor-consumidor (con la cola de mensajes y el búfer circular) nos evita el problema así el sistema está sincronizada.

Permite que los mineros sigan depositando bloques en la cola sin quedarse bloqueados de inmediato, y a su vez, permite que el `Comprobador` deposite los bloques verificados en la memoria compartida de forma asíncrona respecto al `Monitor`. Sin este sistema, la sincronización obligaría a los mineros a detener su ejecución hasta que el `Comprobador` y el `Monitor` terminasen sus tareas y validación.

- **¿Y si el retraso de Comprobador es muy inferior al de Minero? ¿Por qué?**

Sí, sigue siendo necesario, aunque en este caso el problema es menor. Si el `Comprobador` es muy rápido, el búfer circular estará vacío la mayor parte del tiempo, ya que consumirá los bloques casi al instante.

Sin embargo, el modelo productor-consumidor es importante, ya que garantiza la exclusión mutua (evitando condiciones de carrera al acceder a la memoria compartida) y proporciona sincronización condicional (bloqueando al `Monitor` limpiamente cuando no hay bloques, en lugar de hacer espera activa consumiendo CPU).

- ¿Se simplificaría la estructura global del sistema si entre Comprobador y Monitor hubiera también una cola de mensajes? ¿Por qué?

Sí, la respuesta es definitiva. Las colas de mensajes POSIX (`mq_open`, `mq_send`, `mq_receive`) que ya implementan internamente con el modelo productor-consumidor.

Ya que es el SO quien gestiona la memoria del búfer interno, el control de concurrencia (`mutex`) y los bloqueos por cola vacía o llena. Al usar memoria compartida pura (`shm_open`), como hemos hecho en esta práctica, hemos tenido que gestionar manualmente estos recursos y coordinar con los semáforos (`mutex`, `empty`, `full`), complicando la lógica de los códigos.

5. Otros aspectos

En nuestro proyecto hemos usado herramientas como `makefile`, `valgrind`, etc para facilitar el desarrollo de proyecto.

- El programa compila sin errores utilizando el comando `make all` proporcionado en el `Makefile`.
- Todo el código fuente cuenta con comentarios enriquecidos (Doxygen style), explicando las estructuras de datos y la lógica de IPC.
- Se ha comprobado que no existen fugas de memoria (usando `valgrind` con `make valgrind_monitor` y `make valgrind_minero`) ni recursos IPC huérfanos tras la finalización normal del último minero.
- Se puede limpiar los restos de ficheros con comando `make clean`.